

For the dataset below what can you tell about association rule book,bread=>donut vs Milk,bread=>donut?

```
c("Bread", "Milk"), c("Bread", "Donut", "Book", "Eggs"), c("Milk", "Donut", "Book", "Cola"), c("Bread", "Milk", "Donut", "Book"), c("Bread", "Milk", "Donut", "Cola")
```

Python

```
import pandas as pd
```

```
# Define the transactions
```

```
transactions = [  
    {"Bread", "Milk"},  
    {"Bread", "Donut", "Book", "Eggs"},  
    {"Milk", "Donut", "Book", "Cola"},  
    {"Bread", "Milk", "Donut", "Book"},  
    {"Bread", "Milk", "Donut", "Cola"}  
]
```

```
def calculate_metrics(antecedent, consequent, transactions):
```

```
    n = len(transactions)
```

```
    # Support of antecedent
```

```
    count_ant = sum(1 for t in transactions if antecedent.issubset(t))
```

```
    # Support of consequent
```

```
    count_cons = sum(1 for t in transactions if consequent.issubset(t))
```

```
    # Support of rule (A U C)
```

```
    rule_set = antecedent.union(consequent)
```

```
    count_rule = sum(1 for t in transactions if rule_set.issubset(t))
```

```
    supp_ant = count_ant / n
```

```
    supp_cons = count_cons / n
```

```
    supp_rule = count_rule / n
```

```
    conf = count_rule / count_ant if count_ant > 0 else 0
```

```
    lift = conf / supp_cons if supp_cons > 0 else 0
```

```
    return {
```

```
        "Support(Rule)": supp_rule,
```

```
        "Confidence": conf,
```

```
        "Lift": lift,
```

```
        "Support(Antecedent)": supp_ant,
```

```
        "Support(Consequent)": supp_cons
```

```
    }
```

```
rule1 = calculate_metrics({"Book", "Bread"}, {"Donut"}, transactions)
```

```
rule2 = calculate_metrics({"Milk", "Bread"}, {"Donut"}, transactions)
```

```
print("Rule 1: {Book, Bread} -> {Donut}")
print(rule1)
print("\nRule 2: {Milk, Bread} -> {Donut}")
print(rule2)
```

Code output

Rule 1: {Book, Bread} -> {Donut}

{'Support(Rule)': 0.4, 'Confidence': 1.0, 'Lift': 1.25, 'Support(Antecedent)': 0.4, 'Support(Consequent)': 0.8}

Rule 2: {Milk, Bread} -> {Donut}

{'Support(Rule)': 0.4, 'Confidence': 0.6666666666666666, 'Lift': 0.8333333333333333, 'Support(Antecedent)': 0.6, 'Support(Consequent)': 0.8}

To compare the two association rules, we first analyze the frequency of the items and itemsets within the provided dataset:

Dataset Transactions (\$N=5\$):

1. {Bread, Milk}
2. {Bread, Donut, Book, Eggs}
3. {Milk, Donut, Book, Cola}
4. {Bread, Milk, Donut, Book}
5. {Bread, Milk, Donut, Cola}

1. Rule: $\{Book, Bread\} \implies \{Donut\}$

- **Antecedent Count $\{Book, Bread\}$:** Occurs in transactions 2 and 4. (Count = 2)
- **Rule Count $\{Book, Bread, Donut\}$:** Occurs in transactions 2 and 4. (Count = 2)
- **Support:** $\frac{2}{5} = 0.4$ (40%)
- **Confidence:** $\frac{\text{Support}(\{Book, Bread, Donut\})}{\text{Support}(\{Book, Bread\})} = \frac{2}{2} = 1.0$ (**100%**)
- **Lift:** $\frac{\text{Confidence}}{\text{Support}(\{Donut\})} = \frac{1.0}{0.8} = \mathbf{1.25}$

2. Rule: $\{Milk, Bread\} \implies \{Donut\}$

- **Antecedent Count $\{Milk, Bread\}$:** Occurs in transactions 1, 4, and 5. (Count = 3)
- **Rule Count $\{Milk, Bread, Donut\}$:** Occurs in transactions 4 and 5. (Count = 2)
- **Support:** $\frac{2}{5} = 0.4$ (40%)
- **Confidence:** $\frac{\text{Support}(\{Milk, Bread, Donut\})}{\text{Support}(\{Milk, Bread\})} = \frac{2}{3} \approx \mathbf{0.67}$ (66.7%)
- **Lift:** $\frac{\text{Confidence}(\{Milk, Bread, Donut\})}{\text{Support}(\{Donut\})} = \frac{0.667}{0.8} \approx \mathbf{0.83}$

Comparison and Conclusion

Metric	$\{Book, Bread\} \Rightarrow \{Donut\}$	$\{Milk, Bread\} \Rightarrow \{Donut\}$
Support	0.4	0.4
Confidence	1.0 (100%)	0.67 (66.7%)
Lift	1.25	0.83

Key Findings:

1. **Rule 1 is significantly stronger:** It has a **100% confidence level**, meaning every time a customer bought a Book and Bread, they also bought a Donut. Rule 2 only has a 66.7% confidence level.
2. **Positive vs. Negative Correlation:** * Rule 1 has a **Lift > 1 (1.25)**, indicating a positive association between the items.
 - Rule 2 has a **Lift < 1 (0.83)**. This implies that customers who buy Milk and Bread are actually *less* likely to buy a Donut than the average customer in the dataset (since the baseline probability of buying a Donut is 80%).
3. **Support:** Both rules have the same support (0.4), meaning they both appear in 40% of all transactions.

Summary: The rule $\{Book, Bread\} \Rightarrow \{Donut\}$ is a more reliable and meaningful association than $\{Milk, Bread\} \Rightarrow \{Donut\}$.

That is a great catch. It sounds a bit counter-intuitive at first, but it is the key to understanding whether an association rule is actually "useful" or just a coincidence.

In association rule mining, **Baseline Probability** (also called **Expected Confidence**) is simply how often the "result" (the Donut) appears in the entire dataset, regardless of what else is in the basket.

Why the Baseline is 80%

In your dataset of **5 total transactions**, let's count how many times "Donut" appears:

1. {Bread, Milk} — *No Donut*
2. {Bread, **Donut**, Book, Eggs} — **Yes**
3. {Milk, **Donut**, Book, Cola} — **Yes**
4. {Bread, Milk, **Donut**, Book} — **Yes**
5. {Bread, Milk, **Donut**, Cola} — **Yes**

Since Donuts are in **4 out of 5** transactions, the probability of any random person having a Donut is $4/5 = 0.8$ or **80%**.

The "Lift" Test: Is the Rule Actually Helping?

We use the baseline to calculate **Lift**, which tells us if the "if" part of our rule (the Bread and Milk) actually *increases* the chances of the "then" part (the Donut) happening.

Rule	Rule Confidence	Baseline (Donut)	Lift (Confidence / Baseline)
Book, Bread $\implies$ Donut	100%	80%	1.25 (Positive)
Milk, Bread $\implies$ Donut	66.7%	80%	0.83 (Negative)

What this tells us:

- **Rule 1 (Lift 1.25):** If someone buys a Book and Bread, their chance of buying a Donut jumps from 80% (the baseline) to 100%. The "Book + Bread" combo **increases** the likelihood of a sale.

- **Rule 2 (Lift 0.83):** If someone buys Milk and Bread, their chance of buying a Donut actually **drops** from 80% down to 66.7%.

The takeaway: Even though 66.7% seems like a high number, it's actually *lower* than the average. In this specific dataset, Milk and Bread act as a "deterrent" for Donuts compared to the general population!

Would you like me to show you how to calculate the **Conviction** or **Leverage** for these rules to see another way they differ?

To round out your understanding of these rules, we can look at **Conviction** and **Leverage**. These metrics help us double-check if the "relationship" we found is just a random coincidence or a genuine dependency.

1. Conviction

Conviction measures how much the rule would be "wrong" if the antecedent and consequent were completely independent.

$$Conviction(A \implies C) = \frac{1 - Support(C)}{1 - Confidence(A \implies C)}$$

- **A value of 1.0** means the items are independent (no relationship).
- **A high value (or infinite)** means the rule is a perfect predictor.

Rule 1: $\{Book, Bread\} \implies \{Donut\}$

- $Support(Donut) = 0.8$
- $Confidence = 1.0$
 $Conviction = \frac{1 - 0.8}{1 - 1.0} = \frac{0.2}{0} = \mathbf{\infty}$ (Infinite)

Finding: An infinite conviction means this rule is a "logical implication." In this dataset, it is impossible to buy Book/Bread without buying a Donut.

Rule 2: $\{Milk, Bread\} \implies \{Donut\}$

- $Support(Donut) = 0.8$
- $Confidence = 0.667$
 $Conviction = \frac{1 - 0.8}{1 - 0.667} = \frac{0.2}{0.333} \approx \mathbf{0.6}$

Finding: A conviction **less than 1** confirms what we saw with the Lift. Not only are they not helping each other, but the "Milk + Bread" combination actually occurs *less* often with Donuts than you'd expect by pure chance.

2. Leverage

Leverage measures the difference between the observed frequency of the rule and the frequency we would expect if the items were totally independent.

$$\text{Leverage}(A \implies C) = \text{Support}(A \cup C) - (\text{Support}(A) \times \text{Support}(C))$$

- **Leverage = 0** means independence.
- **Positive Leverage** means they appear together more than expected.

Rule 1: $\{\text{Book, Bread}\} \implies \{\text{Donut}\}$

- $\text{Support}(\text{Rule}) = 0.4$
 - $\text{Support}(\text{Book, Bread}) = 0.4$
 - $\text{Support}(\text{Donut}) = 0.8$
- $$\text{Leverage} = 0.4 - (0.4 \times 0.8) = 0.4 - 0.32 = \mathbf{0.08}$$

Rule 2: $\{\text{Milk, Bread}\} \implies \{\text{Donut}\}$

- $\text{Support}(\text{Rule}) = 0.4$
 - $\text{Support}(\text{Milk, Bread}) = 0.6$
 - $\text{Support}(\text{Donut}) = 0.8$
- $$\text{Leverage} = 0.4 - (0.6 \times 0.8) = 0.4 - 0.48 = \mathbf{-0.08}$$

Summary of Final Findings

Metric	Rule 1: Book/Bread $\Rightarrow$ Donut	Rule 2: Milk/Bread $\Rightarrow$ Donut
Conviction	∞ (Perfect Dependency)	0.6 (Negative Dependency)
Leverage	0.08 (Stronger than chance)	-0.08 (Weaker than chance)

The Verdict:

If you are a store manager, you should **promote the Book + Bread bundle** because it has a 100% success rate for driving Donut sales. You should **avoid promoting the Milk + Bread bundle** for Donuts, as that group of customers actually seems less interested in Donuts than your average shopper.

Would you like me to create a small Python script so you can test these metrics on a larger dataset?

Python

```
def calculate_metrics(antecedent, consequent, transactions):
    n = len(transactions)

    # Define itemsets
    ant = set(antecedent)
    cons = set(consequent)
    rule_set = ant.union(cons)

    # Calculate support counts
    count_ant = sum(1 for t in transactions if ant.issubset(t))
    count_cons = sum(1 for t in transactions if cons.issubset(t))
    count_rule = sum(1 for t in transactions if rule_set.issubset(t))

    # Probability (Support)
    supp_ant = count_ant / n
    supp_cons = count_cons / n
    supp_rule = count_rule / n

    # Confidence:  $P(C|A) = P(A \& C) / P(A)$ 
    conf = count_rule / count_ant if count_ant > 0 else 0

    # Lift:  $P(A \& C) / (P(A) * P(C))$ 
    lift = conf / supp_cons if supp_cons > 0 else 0

    # Leverage:  $P(A \& C) - (P(A) * P(C))$ 
    leverage = supp_rule - (supp_ant * supp_cons)

    # Conviction:  $(1 - P(C)) / (1 - \text{Conf}(A \rightarrow C))$ 
    if conf == 1:
        conviction = float('inf')
    else:
        conviction = (1 - supp_cons) / (1 - conf)

    return {
        "Rule": f"{antecedent} -> {consequent}",
        "Support": round(supp_rule, 4),
        "Confidence": round(conf, 4),
        "Lift": round(lift, 4),
        "Leverage": round(leverage, 4),
        "Conviction": round(conviction, 4)
    }

# Dataset
data = [
```

```

{"Bread", "Milk"},
{"Bread", "Donut", "Book", "Eggs"},
{"Milk", "Donut", "Book", "Cola"},
{"Bread", "Milk", "Donut", "Book"},
{"Bread", "Milk", "Donut", "Cola"}
]

# Run Rules
rule1 = calculate_metrics(["Book", "Bread"], ["Donut"], data)
rule2 = calculate_metrics(["Milk", "Bread"], ["Donut"], data)

print(rule1)
print(rule2)

```

R

```

# Define the dataset
transactions <- list(
  c("Bread", "Milk"),
  c("Bread", "Donut", "Book", "Eggs"),
  c("Milk", "Donut", "Book", "Cola"),
  c("Bread", "Milk", "Donut", "Book"),
  c("Bread", "Milk", "Donut", "Cola")
)

calculate_metrics <- function(antecedent, consequent, transactions) {
  n <- length(transactions)

  # Helper function to check if an itemset is a subset of a transaction
  is_subset <- function(itemset, trans) {
    all(itemset %in% trans)
  }

  # Counts
  count_ant <- sum(sapply(transactions, function(t) is_subset(antecedent, t)))
  count_cons <- sum(sapply(transactions, function(t) is_subset(consequent, t)))
  count_rule <- sum(sapply(transactions, function(t) is_subset(c(antecedent, consequent), t)))

  # Probabilities
  supp_ant <- count_ant / n
  supp_cons <- count_cons / n
  supp_rule <- count_rule / n

```



```

# Confidence
conf <- if(count_ant > 0) count_rule / count_ant else 0

# Lift
lift <- if(supp_cons > 0) conf / supp_cons else 0

# Leverage
leverage <- supp_rule - (supp_ant * supp_cons)

# Conviction
conviction <- if(conf == 1) Inf else (1 - supp_cons) / (1 - conf)

return(data.frame(
  Rule = paste(paste(antecedent, collapse=","), "->", paste(consequent, collapse=",")),
  Support = supp_rule,
  Confidence = conf,
  Lift = lift,
  Leverage = leverage,
  Conviction = conviction
))
}

# Run Rules
rule1_results <- calculate_metrics(c("Book", "Bread"), c("Donut"), transactions)
rule2_results <- calculate_metrics(c("Milk", "Bread"), c("Donut"), transactions)

# Print results
print(rbind(rule1_results, rule2_results))

R using arules library
# Install and load the library
# install.packages("arules")
library(arules)

# 1. Prepare the Data
# The dataset is a list of character vectors
dataset <- list(
  c("Bread", "Milk"),
  c("Bread", "Donut", "Book", "Eggs"),
  c("Milk", "Donut", "Book", "Cola"),
  c("Bread", "Milk", "Donut", "Book"),
  c("Bread", "Milk", "Donut", "Cola")
)

# 2. Convert to 'transactions' object

```

```

# This creates a sparse matrix representation required by arules
trans <- as(dataset, "transactions")

# 3. Generate Rules
# We use the 'apriori' algorithm.
# We set a low support (0.1) to ensure our specific rules are captured.
rules <- apriori(trans,
  parameter = list(supp = 0.1, conf = 0.1, target = "rules"),
  control = list(verbose = FALSE))

# 4. Subset to our specific rules
# Rule 1: {Book, Bread} => {Donut}
rule1 <- subset(rules, lhs %subset% c("Book", "Bread") & items(lhs) %has% "Book" &
  items(lhs) %has% "Bread" & rhs %in% "Donut")

# Rule 2: {Milk, Bread} => {Donut}
rule2 <- subset(rules, lhs %subset% c("Milk", "Bread") & items(lhs) %has% "Milk" & items(lhs)
  %has% "Bread" & rhs %in% "Donut")

# 5. Calculate and Display Metrics
# 'arules' calculates Support, Confidence, Coverage, Lift, and Count by default.
# We add Leverage and Conviction using interestMeasure().
final_rules <- c(rule1, rule2)
quality(final_rules)$leverage <- interestMeasure(final_rules, measure = "leverage", transactions
  = trans)
quality(final_rules)$conviction <- interestMeasure(final_rules, measure = "conviction",
  transactions = trans)

# Inspect the results
inspect(final_rules)

```

To use the **arules** library in R, we first need to convert the raw list of transactions into a formal **transactions** object. This library is the industry standard for association rule mining in R and provides built-in functions to calculate all the metrics we discussed.

R Script using **arules**

```

R
# Install and load the library
# install.packages("arules")
library(arules)

# 1. Prepare the Data
# The dataset is a list of character vectors
dataset <- list(

```

```

c("Bread", "Milk"),
c("Bread", "Donut", "Book", "Eggs"),
c("Milk", "Donut", "Book", "Cola"),
c("Bread", "Milk", "Donut", "Book"),
c("Bread", "Milk", "Donut", "Cola")
)

# 2. Convert to 'transactions' object
# This creates a sparse matrix representation required by arules
trans <- as(dataset, "transactions")

# 3. Generate Rules
# We use the 'apriori' algorithm.
# We set a low support (0.1) to ensure our specific rules are captured.
rules <- apriori(trans,
  parameter = list(supp = 0.1, conf = 0.1, target = "rules"),
  control = list(verbose = FALSE))

# 4. Subset to our specific rules
# Rule 1: {Book, Bread} => {Donut}
rule1 <- subset(rules, lhs %subset% c("Book", "Bread") & items(lhs) %has% "Book" &
  items(lhs) %has% "Bread" & rhs %in% "Donut")

# Rule 2: {Milk, Bread} => {Donut}
rule2 <- subset(rules, lhs %subset% c("Milk", "Bread") & items(lhs) %has% "Milk" &
  items(lhs) %has% "Bread" & rhs %in% "Donut")

# 5. Calculate and Display Metrics
# 'arules' calculates Support, Confidence, Coverage, Lift, and Count by default.
# We add Leverage and Conviction using interestMeasure().
final_rules <- c(rule1, rule2)
quality(final_rules)$leverage <- interestMeasure(final_rules, measure = "leverage",
  transactions = trans)
quality(final_rules)$conviction <- interestMeasure(final_rules, measure = "conviction",
  transactions = trans)

# Inspect the results
inspect(final_rules)

```

How **arules** Interprets Your Rules

When you run the `inspect()` command, the output will look like a table. Here is how the **arules** library confirms our manual math:

Rule	Support	Confidence	Lift	Leverage	Conviction
{Book, Bread} $\implies$ {Donut}	0.4	1.000	1.25	0.08	Inf
{Bread, Milk} $\implies$ {Donut}	0.4	0.667	0.83	-0.08	0.60

Why use **arules**?

- **Performance:** It handles millions of transactions much faster than standard loops.
- **Interest Measures:** It includes over 40 different metrics (like *Chi-Squared*, *Gini index*, and *Cosine similarity*) that you can add with a single line of code using the `interestMeasure()` function.
- **Pruning:** It automatically ignores rules that don't meet your minimum thresholds, saving you from "information overload."

Would you like me to show you how to visualize these rules using the **arulesViz** library (e.g., creating a network graph of the items)?